

Processes in Linux

A **process** in Linux is a running instance of a program.

It consists of the **program's instructions, data read from files, other programs, or input from a system user.**

Each process is uniquely identified by a **Process ID (PID).**

Types of Processes

There are two main types of processes in Linux:

1. **Foreground Processes:**

- These are interactive **processes initiated by the user.**
- They **receive input from the keyboard** and send **output to the screen.**
- For example, running the **pwd** command in the terminal.

2. **Background Processes:**

- These are **non-interactive processes** that run without user input.
- They **can be initiated by the system or the user**
- **Allow other processes to run simultaneously.**
- **Eg:** System Services, Web Servers, GUI Services

Managing Processes

Starting a Process

- A process can be started in the **foreground or background.**
- By **default**, processes run in the **foreground.**
- To run a process in the **background, append & to the command:**

```
# Run a process in the background
```

```
$ sleep 100 &
```

Viewing Processes

To view running processes, use the `ps` command:

```
# View all running processes
```

```
$ ps -e
```

```
# View detailed information about all processes
```

```
$ ps -ef
```

The `top` command provides a dynamic real-time view of running processes:

```
# View all running processes in real-time
```

```
$ top
```

Stopping a Process

To stop a **foreground** process, press `Ctrl + C`.

To stop a **background** process, use the `kill` command with the **process ID**:

```
# Kill a process with PID 1234
```

```
$ kill 1234
```

```
# Forcefully kill a process with PID 1234
```

```
$ kill -9 1234
```

Changing Process Priority

The `nice` command starts a process with a specified priority, while the `renice` command changes the priority of an already running process:

```
# Start a process with a priority of 5
```

```
$ nice -n 5 sleep 100
```

```
# Change the priority of a process with PID 1234 to 10
```

```
$ renice 10 1234
```

Special Types of Processes

- **Daemon Processes:**

- These are background processes that start at system startup and run continuously.
- They often run with root permissions and service requests from other processes.

- **Zombie Processes:**

- These are processes that have completed execution but still have an entry in the process table.

- **Orphan Processes:**

- These are child processes whose parent process has terminated.
- The **init** process adopts them.

Understanding and managing processes is crucial for maintaining a healthy Linux system. By using the commands and techniques mentioned above, you can effectively monitor and control processes on your system.

Process- Parent and Child processes

- Running program is a **process**.
- From this process, another process can be created.
- There is a **parent-child relationship between the two processes**.
- This can be achieved using a library function called **fork()**.
- fork() function **splits the running process into two processes**.
- The **existing** one is known as **parent** and the **new process** is known as a **child**.

Process: A process is a program under execution i.e an active program. A process is more than the program code, it includes the following:

1. Program counter.
2. Process stack.
3. Registers.
4. Program code, etc.

- A process **changes its state as it executes**.
- The **new state partially depends on the current activity of a process**.
- The different states of the process during its execution are:

1. New
2. Ready
3. Running
4. Blocked
5. Terminated.

A **process control block and process table** are associated with each of the processes.

It contains the following important information about the process:

1. Process state.
2. Process number.
3. Program counter.
4. List of files and registers.

5. CPU information.
6. Memory information, etc.

Parent Process:

All the processes are created when a process executes the `fork()` system call except the startup process.

The process that executes the `fork()` system call is the **parent process**.

A parent process is one that creates a **child process** using a `fork()` system call.

A **parent process may have multiple child processes**, but a **child process only one parent process**.

On the **success** of a `fork()` system call:

- The **Process ID (PID)** of the child process is returned to the parent process.
- **0** is returned to the child process.

On the **failure** of a `fork()` system call,

- **-1** is returned to the parent process.
- A child process is not created.

Child Process: A child process is created by a parent process in an operating system using a `fork()` system call. A **child process may also be known as subprocess or a subtask**.

- A child process is **created as a copy of its parent process**.
- The child process **inherits most of its attributes**.
- **If a child process has no parent process, then the child process is created directly by the kernel.**

- If a child process exits or is interrupted, then a **SIGCHLD signal** is sent to the parent process to inform about the termination or exit of the child process.

Why Do We Need to Create A Child Process?

Sometimes there is a need for a program to perform more than one function **simultaneously**. Since these **jobs may be interrelated** so two different programs to perform them cannot be created.

For example: Suppose there are two jobs: copy contents of source file to target file and display an animated progress bar indicating that the file copy is in progress. The GIF progress bar file should continue to play till file copy is taking place. Once the copying process is finished the playing of the GIF progress bar file should be stopped. Since both these jobs are interrelated they cannot be performed in two different programs. Also, they cannot be performed one after another. Both jobs should be performed simultaneously.

At such times **fork()** is used to create a child process and then write the program in such a manner that file copy is done by the parent and displaying of the animated GIF file is done by the **child process**.

Advantages of using parent and child processes in modern operating systems include:

1. **Concurrency:** By allowing multiple processes to run concurrently, the system **can make better use of available resources such as CPU time, memory, and disk I/O**.
2. **Isolation:** Each process has its own memory space, program code, and **data**, which provides **a degree of isolation and protection** from other processes in the system.

3. **Fault tolerance:** If a child process fails, the parent process can detect the failure and take appropriate action, such as restarting the child process or terminating the program.
4. **Modularity:** By breaking down a program into smaller, independent processes, it becomes easier to modify, maintain, and test each process separately.

However, there are also some disadvantages of using parent and child processes:

1. **Overhead:** Creating and managing multiple processes requires additional system resources, such as memory, CPU time, and disk I/O, which can slow down the system.
2. **Communication overhead:** Interprocess communication (IPC) can be a complex and time-consuming task, particularly when large amounts of data need to be transferred between processes.
3. **Synchronization:** Coordination between processes can be difficult and requires careful design to avoid issues such as deadlocks and race conditions.
4. **Complexity:** Designing and debugging a program that uses multiple processes can be more complex than designing a single-threaded program, and may require additional skills and expertise.
5. Overall, the benefits of using parent and child processes usually outweigh the disadvantages, particularly in systems that require high levels of concurrency, fault tolerance, and modularity.